□ 서식4_한이음 드림업 프로젝트 결과보고서

2026년 한이음 드림업

프로젝트 결과보고서

프로젝트명	공공데이터 AI 데이터셋 자동 표준화 및 검증 플랫폼 개발

요 약 본

프로젝트 정보	
프로젝트명	공공데이터 AI 데이터셋 자동 표준화 및 검증 플랫폼 개발
주제 영역	☐ 생활 ☐ 업무 ■ 공공/교통 ☐ 금융/핀테크 ☐ 의료 ☐ 교육 ☐ 유통/쇼핑 ☐ 엔터테인먼트
기술 분야	■ SW·AI ☐ 방송·콘텐츠 ☐ 블록체인·융합 ☐ 디바이스 ☐ 차세대보안 ☐ 미래통신·전파
달성 성과	☐ 논문게재 및 포스터 발표 ☐ 앱등록 ☐ 프로그램등록 ☐ 특허 ☐ 기술이전 ☐ 실용화 ☐ 공모전( ) ☐ 기타( ) 해당 성과 체크
프로젝트 소개	• 공공데이터를 수집·정제·검증해 분석과 AI 학습에 바로 쓸 수 있는 테이블과 데이터셋으로 만드는 플랫폼 • 실행 엔진 KPubData Builder와 웹 작업공간 KPubData Studio로 구성 • 공공 API 접근은 자체 라이브러리 kpubdata가 담당
개발 배경 및 필요성	• 공공데이터는 개방되어 있지만 기관마다 형식이 달라 바로 분석하기 어려움 • 정제 규칙과 처리 이력이 흩어지면 같은 결과를 다시 만들거나 오류 원인을 찾기 힘듦 • 규칙을 명세로 선언하고 실행 결과를 기록하는 도구가 필요
프로젝트 특·장점	• 명세(BuildSpec) 하나로 수집부터 내보내기까지 선언하고 실행 전에 검증 • 실행마다 출처·행 수·품질 결과를 Manifest로 남기고 테이블은 스냅샷으로 관리 • 코드 없이 웹 화면에서 테이블 생성, SQL 분석, 리포트 작성 가능
주요 기능	• 소스 카탈로그 탐색과 3단계 테이블 만들기 • Bronze·Silver·Gold 단계 처리, 품질 검사, 개인정보 검사와 마스킹 • 스냅샷 기반 SQL 분석, 리포트, 다중 포맷 내보내기와 Hugging Face 게시
기대효과 및 활용 분야	• 공공데이터 가공 과정을 누구나 같은 방식으로 반복하고 설명할 수 있음 • 공공데이터 기반 수업·연구, 데이터 분석, AI 학습용 데이터셋 구축과 배포에 활용

(본문) 프로젝트 결과보고서

Ⅰ. 프로젝트 개요

1. 프로젝트 소개

1) 프로젝트 정의

- 공공데이터를 분석 가능한 테이블로 만들고 그 과정을 확인하는 플랫폼
- 사용자가 규칙을 선언하면 수집, 정제, 검증, 패키징, 기록까지 자동으로 수행
- 만들어진 테이블은 스냅샷으로 보관되어 SQL 분석, 리포트, 내보내기, 게시에 사용
- 기획 의도: 공공데이터를 '받아오는 일'과 '쓸 수 있게 만드는 일' 사이의 간격을 줄이는 것

2) 구성 요소

구성 요소	형태	역할
KPubData Builder	Python 패키지, HTTP 서비스, CLI	명세 검증, Bronze·Silver·Gold 실행, 테이블 스냅샷과 Manifest 기록, 읽기 전용 SQL, 내보내기와 게시
KPubData Studio	React 웹 애플리케이션	카탈로그 탐색, 테이블 만들기, 테이블 조회, SQL 분석, 리포트, 실행·품질·운영 확인
kpubdata	Python 라이브러리	제공 기관별 공공 API 호출과 표준 레코드 변환. Builder가 가져다 씀
BuildSpec	YAML·JSON 명세	소스, 스키마 계약, 품질 기준, 분할, 내보내기를 선언하는 실행 입력
Manifest	manifest.json	실행마다 남는 결과 기록: 상태, 출처, 행 수, 품질 결과, 실패 내역
웨어하우스	Parquet 스냅샷 + SQLite 카탈로그	성공한 실행의 결과를 읽기 전용 스냅샷으로 보관

3) 사용 흐름

- ① 카탈로그에서 소스 데이터셋을 찾고 '시작'을 누름
- ② 테이블 만들기 마법사에서 파라미터와 규칙을 입력
- ③ 미리보기로 스키마와 샘플을 확인하고 검증을 통과하면 '만들기' 실행
- ④ Builder가 수집부터 스냅샷 커밋까지 수행하는 동안 실행 상세에서 진행 상황 확인
- ⑤ 테이블 상세에서 스키마·품질 확인, SQL 작업 공간에서 분석, 리포트 작성
- ⑥ 필요하면 파일로 내보내거나 Hugging Face에 게시

4) 주요 용어

용어	뜻
소스 데이터셋	제공 기관이 공개한 원천 데이터. 카탈로그에서 찾음
테이블	소스 데이터셋으로 만든 결과물. 스냅샷의 묶음
스냅샷	한 번의 실행이 커밋한 읽기 전용 결과. 테이블마다 최근 3개 보관
실행	BuildSpec 하나를 처리한 한 번의 작업. run_id로 구분
Bronze	원천 레코드를 그대로 보존한 단계
Silver	선언한 규칙으로 정규화하고 검증한 단계
Gold	컬럼 선택·마스킹·분할을 거친 최종 단계

2. 개발 배경 및 필요성

1) 공공데이터 활용 환경

- 생성형 AI 확산으로 출처와 품질이 확인된 학습·분석용 데이터 수요 증가
- AI 학습 데이터 시장은 2024년 약 26억 달러에서 2030년 86억 달러로 성장 전망 (Grand View Research, 2025)
- 국내 공공데이터는 OECD 공공데이터 평가에서 4회 연속 1위. 개방 규모는 크지만 가공은 이용자 몫

2) 기존 방식의 문제

구분	문제
형식 차이	기관마다 응답 형식(XML·JSON), 페이지 처리, 인증 방식, 컬럼 이름이 달라 소스마다 수집 코드를 새로 작성
정제 반복	결측 표기('-', 'N/A'), 숫자의 쉼표, 앞자리 0이 잘린 코드 같은 문제를 매번 손으로 정리
이력 부재	어떤 입력과 규칙으로 결과가 만들어졌는지 남지 않아 같은 결과를 다시 만들기 어려움
품질 확인	중복, 결측, 범위 이탈을 배포 전에 확인하는 절차가 없음
법적 확인	개인정보 포함 여부와 재배포 조건을 사람이 따로 확인해야 함
접근 장벽	스크립트와 명령줄 중심이라 개발자가 아니면 쓰기 어려움

3) 개발 목표

- 수집부터 배포까지의 규칙을 하나의 명세로 선언하고, 실행 전에 검증
- 단계마다 결과를 파일로 남겨 어디서 문제가 생겼는지 확인 가능하게 함
- 품질·개인정보·라이선스 확인을 실행 경로에 포함
- 같은 기능을 웹 화면에서 코드 없이 사용
- 결과 테이블을 스냅샷으로 보관해 분석을 다시 실행해도 같은 입력을 읽게 함

3. 프로젝트 특·장점

1) 기존 방식과의 비교

구분	기존 방식	본 프로젝트
공공데이터 포털	원천 API 제공까지. 가공은 이용자 몫	정제·검증·분할·문서화를 거친 테이블과 데이터셋 산출
범용 ETL 도구 (Airflow 등)	파이프라인은 실행되나 검증 규칙은 직접 작성	스키마 계약과 품질 기준을 명세에 선언하고 실행 전에 검증
데이터 허브 (Hugging Face 등)	배포·공유 중심. 생성 과정은 남지 않음	실행 기록과 입력 지문을 Manifest로 남기고 카드와 함께 게시
LLM 데이터 도구	LLM이 내용을 추정해 생성	답변을 실제 근거와 대조하고 사용자 승인 전에는 실행하지 않음

2) 기술적 차별점

- **명세 기반 실행**
  - BuildSpec이 실행의 유일한 입력. 화면에서 만든 설정도 같은 명세로 변환되어 전달
  - 실행 전 15개 영역을 검사해 문제를 code·path·message로 한 번에 반환
  - 실행에 쓰인 명세를 스냅샷으로 남겨 나중에 그대로 다시 실행 가능
- **단계별 결과 보존**
  - Bronze·Silver·Gold 결과를 각각 파일로 남겨 단계별로 확인
  - 소스 하나가 실패해도 나머지 소스는 계속 처리하고 실패 단계와 코드를 기록
- **스냅샷과 재현성**
  - 테이블은 읽기 전용 스냅샷으로 커밋. 저장한 분석은 읽은 스냅샷과 함께 보관
  - 입력 지문, 분할 알고리즘, 산출물 다이제스트를 Manifest에 기록
- **실행 경로에 포함된 점검**
  - 품질 검사 7종, 개인정보 패턴 검사와 마스킹, 재배포 조건 판정
  - 게시 전 준비 상태 점검: 라이선스, 개인정보 검사 통과, 카드 항목 누락
- **역할 분리**
  - Studio는 화면, Builder는 검증·실행·질의, kpubdata는 공공 API 접근

- Studio는 Builder가 주지 않은 값을 만들어 보여 주지 않음 (점수 대신 검사 결과, 못 받은 값은 '확인 불가')

- **근거에 묶인 AI 도우미**
  - 답변의 참조·동작·SQL을 Builder에서 받은 근거와 대조
  - SQL 실행과 명세 변경은 사용자가 승인한 뒤에만 수행

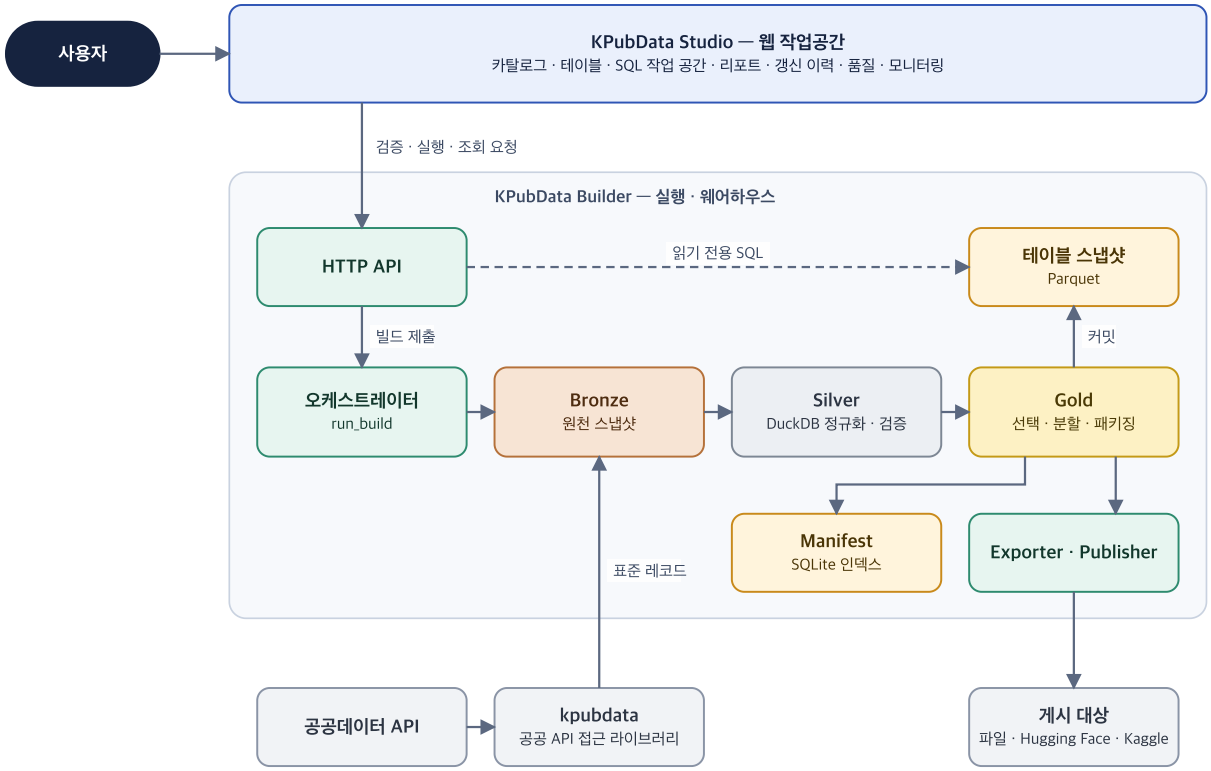
3) 유사 제품 기능 비교 (○ 제공, △ 일부 제공, × 미제공)

기능	본 프로젝트	공공데이터 포털	HF Datasets	Kaggle	dbt·ETL	DVC
공공 API 직접 수집	○	○	×	×	△	×
자동 정규화·표준화	○	×	×	×	○	×
선언적 빌드(명세)	○	×	×	×	○	△
다중 포맷 내보내기	○	△	○	△	×	×
데이터셋 카드 자동 생성	○	×	△	△	×	×
재현성(Manifest·버전)	○	×	△	×	△	○
허브 배포(HF·Kaggle)	○	×	○	○	×	×
비개발자용 웹 UI	○	△	△	○	×	×
국내 공공데이터 특화	○	○	×	×	×	×

Ⅱ. 프로젝트 내용

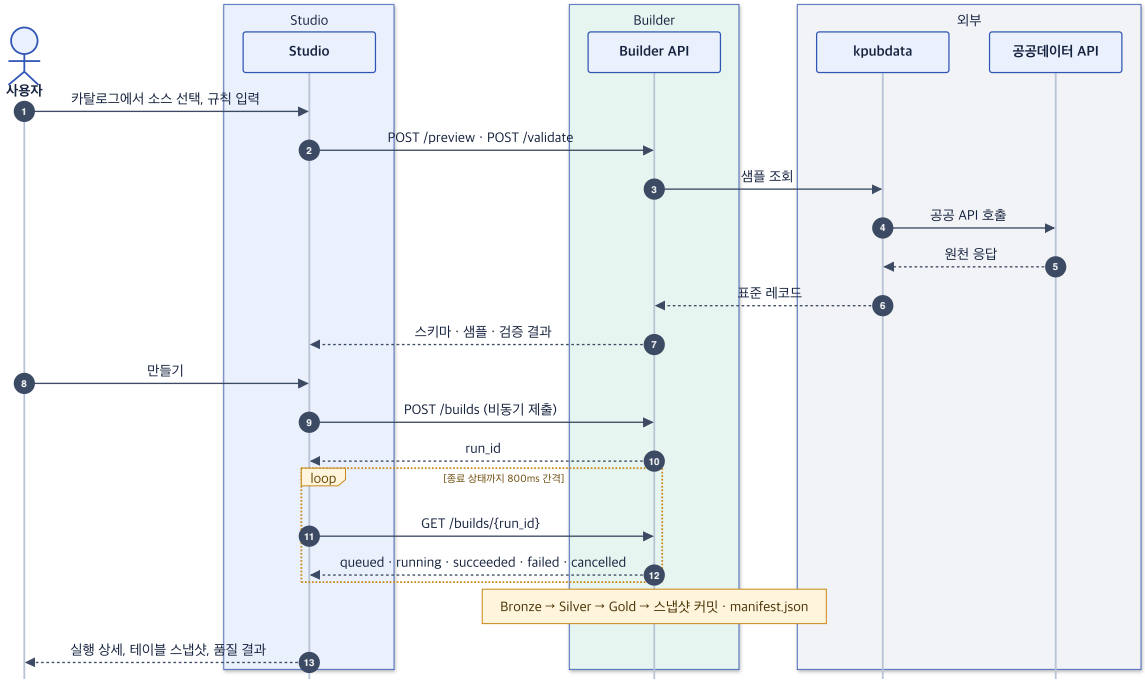
1. 프로젝트 구성도

1) 시스템 구성도



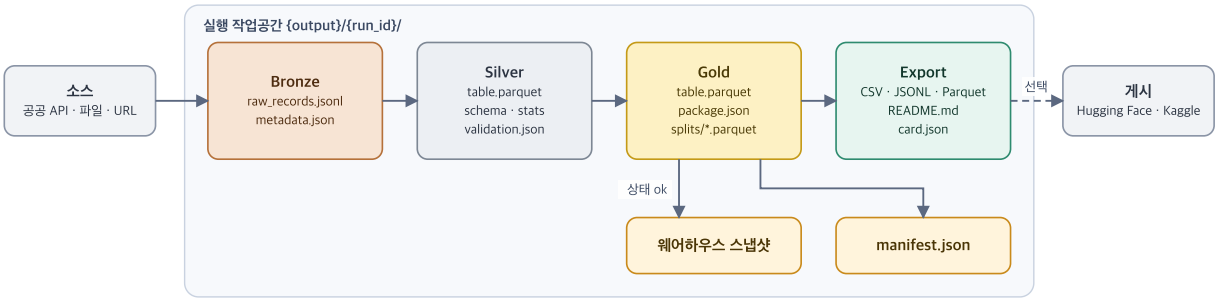
- Studio는 Builder API만 호출하며 실행 엔진을 갖지 않음
- Builder는 kpubdata로 원천을 받아 Bronze → Silver → Gold 순으로 처리
- 성공한 실행의 Gold는 테이블 스냅샷으로 커밋되고 실행 내역은 Manifest와 SQLite 인덱스에 기록
- SQL 질의는 저장된 스냅샷을 읽기 전용으로 조회

2) 서비스 흐름도



- 미리보기와 검증은 저장 없이 수행. 통과한 뒤에만 실행 제출
- 실행은 비동기. Studio는 800ms 간격으로 상태를 조회하고 탭이 숨겨지면 조회를 멈춤
- 작업 상태: queued → running → succeeded · failed · cancelled

3) 데이터 처리 흐름



단계	처리	산출물
Bronze	공공 API·업로드 파일·URL의 원본을 그대로 수집하고 수집 시각·파라미터·건수·체크섬 기록	raw_records.jsonl, metadata.json
Silver	선언한 규칙으로 정규화, 스키마 검증, 개인정보·품질 검사, 통계와 미리보기 생성	table.parquet, schema.json, stats.json, validation.json
Gold	행 필터와 컬럼 선택, 개인정보 컬럼 마스킹, 분할, 두 소스 조인	table.parquet, package.json, splits/*.parquet
Export	지정한 형식으로 내보내고 데이터셋 카드 생성	CSV·JSONL·Parquet·Markdown 파일, README.md, card.json
커밋·기록	상태가 ok이면 웨어하우스 스냅샷으로 커밋하고 Manifest 기록	스냅샷, manifest.json

4) 저장 구조

- 별도 DB 서버 없이 SQLite와 로컬 파일만 사용
- 실행 작업공간: `{output}/{run_id}/` 아래에 bronze/, silver/, gold/, buildspec.yaml, manifest.json
- 웨어하우스: `{warehouse}/tables/{id}/snapshots/{id}/`에 Parquet. 카탈로그는 `_warehouse.sqlite`
- 실행 인덱스(`_builds.sqlite`)는 Manifest에서 다시 만들 수 있는 파생 자료
- 실행 이벤트, 게시 영수증, 저장된 분석, 명세 개정 이력, 자격 증명(암호화)도 각각 SQLite 파일로 보관

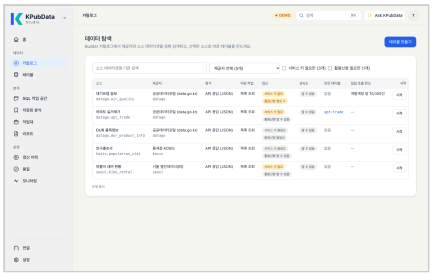
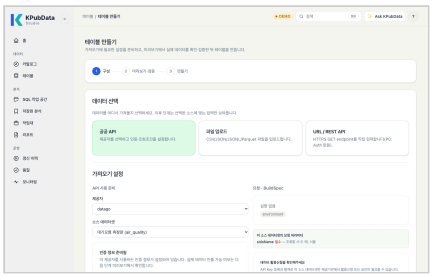
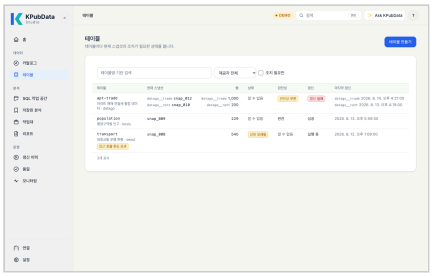
2. 프로젝트 기능

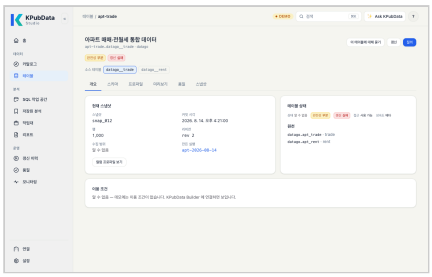
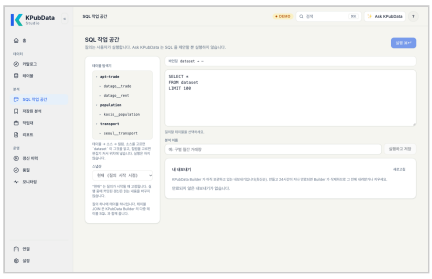
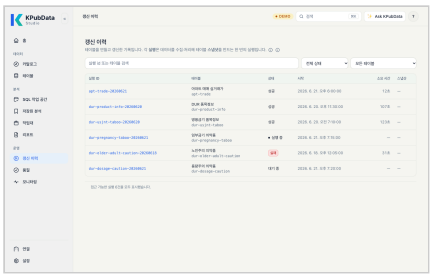
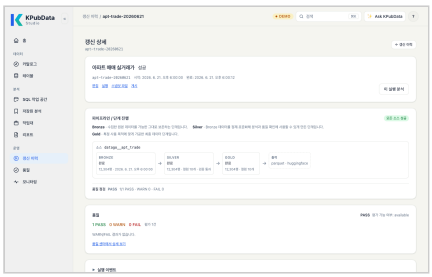
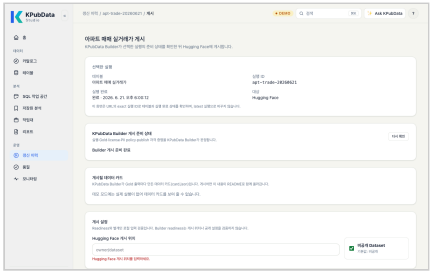
1) 전체 기능 목록

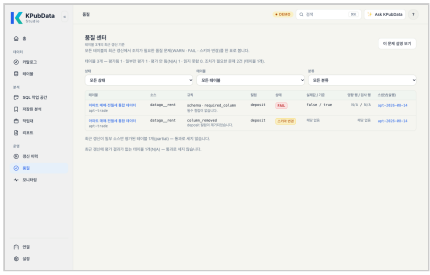
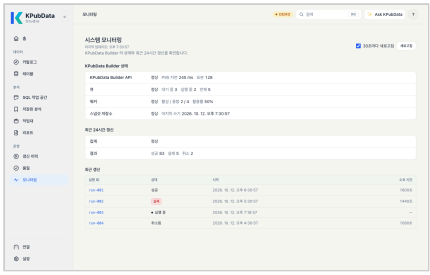
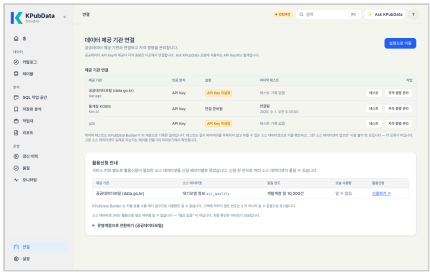
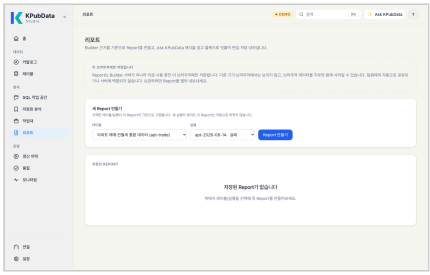
구분	기능	설명
수집	공공 API 수집	kpubdata로 원천 레코드를 페이지 단위로 수집. 파라미터 조합별 체크포인트
	파일·URL 수집	업로드 파일(csv·json·jsonl·parquet), HTTPS URL(json·jsonl·csv)
	소스 카탈로그	제공 기관과 데이터셋 목록, 접근 조건, 호출 한도 조회
	정기 수집	GitHub Actions 일정으로 분야별 데이터 주기 수집·게시
가공·품질	BuildSpec 검증	15개 영역 검사, 문제를 code·path·message로 반환
	미리보기	저장 없이 소스별 스키마와 샘플 행 확인
	Silver 정규화	결측 토큰, 컬럼 병합, 이름 변경, 자리 채움, 형 변환, 파생 컬럼
	품질 검사	필수 컬럼, 자료형, 중복률, 결측률, 최소 행 수, 값 범위, 컬럼 비교
	개인정보 검사·마스킹	주민등록번호·전화번호·이메일·사업자번호 패턴 검사, 선언 컬럼 마스킹
	스키마 변화 감지	이전 실행 대비 컬럼 추가·삭제, 자료형 변경, 행 수 급변 기록
	Gold 선택·분할·조인	행 필터, 컬럼 선택, 비율·키 분할, 두 소스 조인
산출물	테이블 스냅샷	읽기 전용 스냅샷 커밋, revision 관리, 최근 3개 보관
	Manifest	상태·출처·행 수·품질 결과·실패 내역·입력 지문 기록
	내보내기	CSV·JSONL·Parquet·Markdown·Hugging Face·Kaggle 형식
	데이터셋 카드	스키마·샘플·출처·처리 내역·개인정보 항목을 담은 README.md, card.json
	게시	준비 상태 점검 후 Hugging Face 등으로 게시. 영수증과 대조 기능
웹 작업공간	홈	조치가 필요한 테이블, 최근 스냅샷·분석, 연결 문제, 첫 실행 체크리스트
	카탈로그·테이블 만들기	소스 검색·필터, 3단계 마법사, 초안 저장
	테이블·테이블 상세	스냅샷, 스키마, 프로파일, 미리보기, 품질, 스냅샷 이력
	SQL 작업 공간	스냅샷 대상 읽기 전용 SQL, 분석 저장, 집계 차트, 결과 내보내기
	리포트·작업대	근거 기반 리포트 작성과 내려받기, 최근 작업과 저장한 명세
	갱신 이력·실행 상세	실행 목록, 단계별 진행, 이벤트 타임라인, 취소, 재실행
운영·보안	품질 센터·모니터링	테이블 전체의 품질 경고, Builder 상태와 24시간 실행 통계
	연결·설정	제공 기관 자격 증명 관리, 키 확인, Builder 버전 호환 점검
	인증·관리	Keycloak OIDC 로그인, 가입 승인, 전체 실행 조회(관리자)
	읽기 전용 SQL 검사	단일 SELECT만 허용, 테이블 함수·비결정 함수·외부 접근 차단
	재배포 판정	소스별 재배포 조건 중 가장 엄격한 값을 적용, 게시 시 라이선스 필수
AI 보조	Ask KPubData	화면 문맥과 근거를 바탕으로 답변·SQL·다음 동작 제안. 4단계 검증

H/W	해당 없음	전용 하드웨어를 사용하지 않는 소프트웨어 프로젝트

2) S/W 주요 기능

기능	설명	프로젝트실물사진
공공데이터 탐색	• 카탈로그에서 소스 데이터셋 검색 • 제공 기관, 서비스 키·활용 신청 필요 여부로 필터 • 접근 조건과 호출 한도 비교 • 시작 버튼으로 테이블 만들기 진입	
테이블 만들기	• 구성 → 미리보기·검증 → 만들기 3단계 • 공공 API, 파일 업로드, URL 소스 지원 • 미리보기와 검증을 동시에 요청 • 검증을 통과해야 실행 가능	
테이블 목록	• 테이블별 현재 스냅샷과 행 수 • 상태·완전성·갱신 여부 표시 • 조치가 필요한 테이블만 보기	

기능	설명	프로젝트실물사진
테이블 상세	• 개요·스키마·프로파일·미리보기·품질·스냅샷 탭 • 과거 스냅샷 조회와 현재로 돌아가기 • 갱신, 쿼리, 질문으로 이동	
SQL 분석	• 스냅샷을 골라 읽기 전용 SQL 실행 • 분석 저장과 재실행 • 집계 차트, 결과 내보내기	
실행 이력	• 실행을 한 표로 조회 • 상태·테이블 필터와 검색 • 행을 눌러 실행 상세로 이동	
실행 확인	• 소스별 단계 진행과 품질 결과 • 실행 이벤트 타임라인 • 실패 시 단계와 코드 표시, 재실행	
게시	• 게시 준비 상태와 차단 사유 확인 • 데이터 카드 미리보기 • Hugging Face 저장소로 게시	

기능	설명	프로젝트실물사진
품질 확인	• 테이블 전체의 WARN·FAIL 결과 • 규칙, 컬럼, 실제값과 기준값 표시 • 스키마 변화 함께 조회	
운영 확인	• API·대기열·작업자·저장소 상태 • 최근 24시간 실행 통계 • 30초 간격 자동 조회	
연결 관리	• 제공 기관별 인증 방식과 설정 여부 • 키 등록과 확인 • 활용 신청 안내	
리포트	• 테이블과 실행을 골라 리포트 생성 • 근거 영역과 메모 편집 • Markdown·HTML 내려받기	

- 화면은 데모 데이터로 실행한 Studio를 캡처한 것임 실제 운영 화면 확보 시 교체

3) H/W 주요 기능

기능/부품	설명	프로젝트실물사진
해당 없음	-	-

3. 주요 적용 기술

1) 선언형 실행 계약 (BuildSpec)

- 필수 항목은 dataset_id, title, description, sources 네 가지. 나머지는 필요한 것만 선언
- 소스마다 스키마 계약(schema)과 Gold 선택(gold)을 따로 지정
- exports를 비워 두면 테이블 커밋까지만 수행 (내보내기는 선택)

명세 항목	선언 내용
`sources[]`	소스 종류(public_api·file·url), 제공 기관·데이터셋, 수집 파라미터, 스키마 계약, Gold 선택
`quality`	중복률·결측률 상한, 최소 행 수, 값 범위, 컬럼 비교와 규칙별 심각도
`pii`	개인정보 검사 정책 (block·warn·allow)
`splits`	분할 방식(ratio·key), 비율, 시드
`composition`	두 소스의 조인 정의 (키, inner·left, 중복·결측 키 처리)
`exports[]`	내보내기 형식과 경로
`license` 등	라이선스, 출처 표기, 게시 여부, 갱신 주기

- 검증은 실행 자원을 만들기 전에 수행하고 문제를 모두 모아 반환
- 검사 영역: 필수 항목, 소스 식별, 내보내기, 메타데이터, 분할, 스키마 계약, 소스 종류, 소스 키, 파라미터 축, 개인정보 정책, 라이선스, 품질, 조인, Gold 선택

2) Silver 정규화

순서	규칙	동작
1	읽기 형식	지정한 컬럼을 문자열로 읽어 앞자리 0 보존
2	결측 토큰	'-', 'N/A' 같은 값을 결측으로 처리 (전역, 컬럼별)
3	컬럼 병합	이름만 다른 여러 컬럼을 하나로 합침. 값이 다르면 실패
4	이름 변경	원본 필드명을 표준 이름으로 변경
5	자리 채움	지정한 자릿수까지 왼쪽을 0으로 채움
6	형 변환	bool·date·datetime·float·int·str, 천 단위 쉼표 숫자. 값이 사라지면 오류
7	파생 컬럼	연·월·일 컬럼으로 날짜 생성, 여러 컬럼으로 조인 키 생성

- 선언한 규칙만 위 순서로 적용하고, 선언하지 않은 변환은 하지 않음
- 처리는 DuckDB SQL로 수행하고 결과는 Parquet으로 저장

3) 품질 검사

규칙	검사 내용
`required_column · dtype`	필수 컬럼 존재, 컬럼 자료형 일치
`max_duplicate_rate`	중복 행 비율 상한
`max_null_ratio`	컬럼별 결측 비율 상한
`min_rows`	최소 행 수
`range`	숫자 컬럼의 값 범위 (경계 포함)
`compare_columns`	두 컬럼의 대소·동일 관계

- 결과는 pass·warn·fail. fail이 있으면 해당 소스를 Gold 전에 중단, warn은 기록하고 진행
- 평가하지 못한 규칙은 통과로 치지 않고 결과에서 제외

4) 개인정보 검사와 마스킹

- 검사 대상: 주민등록번호(날짜·검증 숫자 확인), 전화번호, 이메일, 사업자등록번호, 개인정보를 뜻하는 컬럼명
- 정책: block은 검출 시 중단, warn은 기록 후 진행, allow는 통과 (게시와 함께 쓸 수 없음)
- 마스킹: 선언한 컬럼의 문자열은 [masked]로, 그 밖의 자료형은 결측으로 변경
- 게시는 개인정보 검사를 block 정책으로 통과한 실행에만 허용 (다중 사용자 배포 기준)

5) 분할

- 비율 분할: 시드와 행 순번의 해시로 정렬한 뒤 비율대로 나눔. 같은 입력이면 항상 같은 결과
- 키 분할: 지정한 컬럼의 값별로 나눔

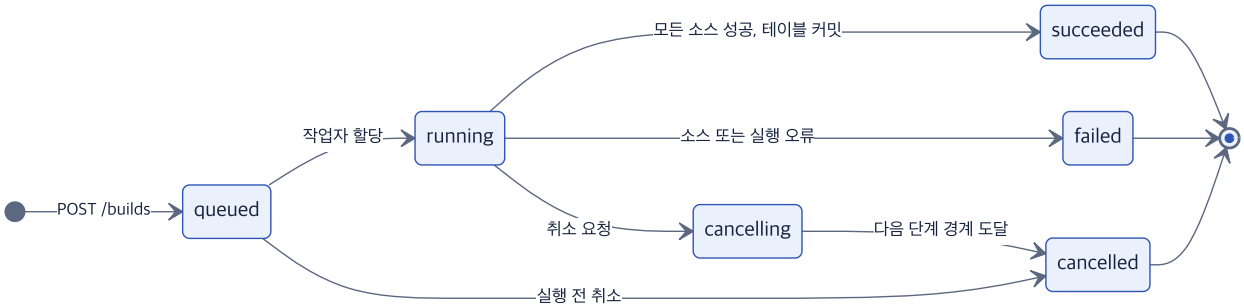
6) 테이블 스냅샷과 동시 갱신 제어

- 상태가 ok인 실행의 Gold만 스냅샷으로 커밋
- 실행 시작 때 테이블의 revision을 기억했다가 커밋할 때 비교. 다르면 커밋을 거부하고 사유 기록
- 거부 사유: 같은 이름의 테이블 존재, 0행 결과, revision 충돌, 커밋 오류
- 이전 스냅샷은 최근 3개까지 보관. 저장된 분석이 쓰는 스냅샷은 정리하지 않음

7) 읽기 전용 SQL 검사

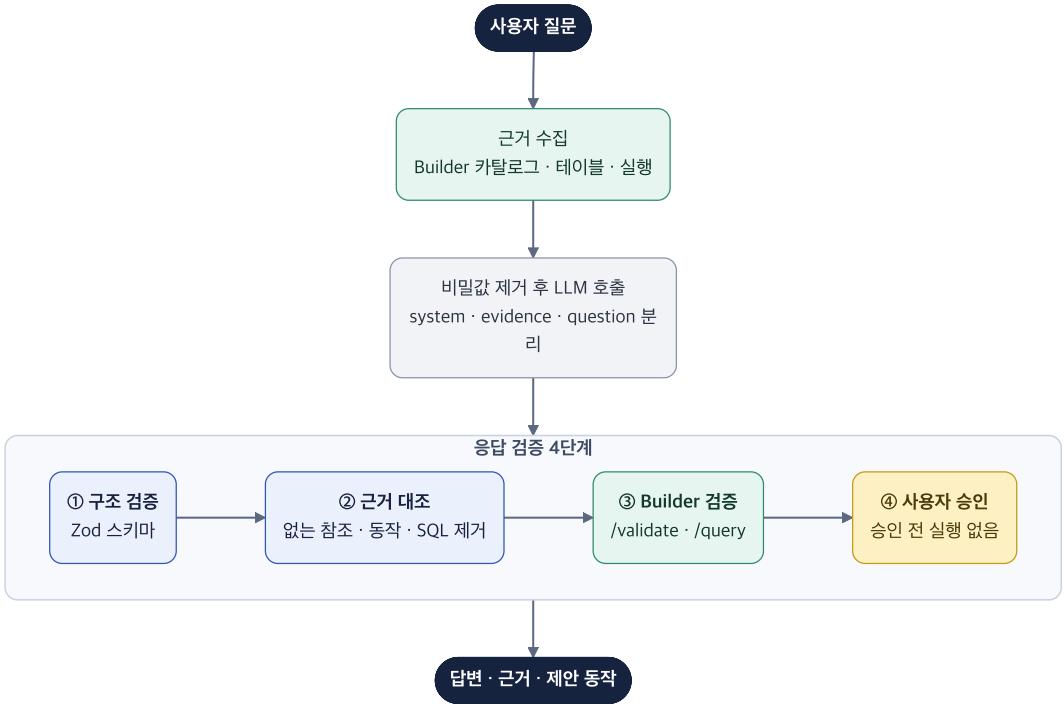
- sqlglot으로 구문을 분석해 SELECT 한 문장만 허용 (DDL·DML, 재귀 CTE 불가)
- 테이블 함수, VALUES·UNNEST·LATERAL, 표본 추출 불가
- 환경을 들여다보는 함수 44종, 결과가 매번 달라지는 함수 18종 불가
- 실제 테이블 참조는 논리 이름 `dataset` 하나만 허용
- 통과한 문장은 표준형으로 다시 만들어 실행. 잠긴 DuckDB 연결과 별도 프로세스가 추가 방어

8) 비동기 실행과 취소



- 실행은 대기열에 넣고 작업자가 처리. 기본 동시 실행 2건, 대기 10건, 실행 시간 6시간
- 취소는 단계 경계에서만 반영하는 협력적 방식. 이미 만든 산출물은 부분 Manifest와 함께 보존
- 소스는 스레드 풀로 최대 4개까지 병렬 처리하고 결과는 명세 순서대로 병합

9) AI 도우미 응답 검증



- 프롬프트를 고정 지시, 신뢰하지 않는 근거, 사용자 질문으로 나누어 전달
- 전송 전에 키·토큰 이름의 필드, key=value 형태, 무작위성이 높은 긴 문자열을 자리표시자로 치환
- ① 구조 검증(Zod) → ② 근거 대조 → ③ Builder 검증 → ④ 사용자 승인
- 허용 동작 6종만 제안 가능. 실행 시작, 게시, 자격 증명 변경, SQL 자동 실행은 불가
- LLM 키는 사용자가 직접 입력하며 기본적으로 메모리에만 보관

10) 인증과 통신

- 로그인: Keycloak 기반 OIDC (Authorization Code + PKCE). 토큰은 메모리에만 보관
- 401 응답 시 토큰을 갱신해 같은 요청을 한 번 다시 보내고, 실패하면 로그인 화면으로 이동
- API 호출: 30초 제한, 연결 실패·시간 초과·5xx에 최대 2회 재시도. 미리보기·제출·게시·쿼리는 재시도하지 않음
- 모든 응답을 Zod 스키마로 검증해 계약과 다르면 오류로 처리
- Builder 계약 버전이 최소 요건(1.96.0)보다 낮으면 모든 화면에 경고 배너 표시
- 제공 기관 키는 Builder에 암호화해 저장하거나(단일 사용자) 세션 동안만 메모리에 보관(다중 사용자)

11) 데모 모드와 다국어

- 기능마다 Builder용과 데모용 클라이언트를 두고 같은 계약 테스트로 검증
- Builder 없이도 고정 데이터로 전체 화면을 확인할 수 있으며 모든 화면에 Demo 배지 표시
- 한국어(기본)와 영어 지원

4. 프로젝트 개발 환경

구분	항목	상세내용
S/W 개발환경	OS	개발 장비 OS 기재 / 배포: Linux 컨테이너 (Docker)
	개발환경(IDE)	사용 IDE 기재
	개발도구	Builder: DuckDB 1.2 이상, pyarrow, SQLite, sqlglot, uv Studio: React 19.3, Vite 8, React Router 7, Zustand 5, Tailwind CSS 4, Zod 4, i18next, keycloak-js
	개발언어	Python 3.10~3.13 (Builder), TypeScript 6 (Studio), Node 22 이상
	기타사항	테스트: pytest·ruff·mypy, Vitest·Playwright·axe-core / 인증: Keycloak / 문서: MkDocs
H/W 구성장비	디바이스	해당 없음
	센서	해당 없음
	통신	해당 없음
	언어	해당 없음
	기타사항	해당 없음
프로젝트 관리환경	형상관리	Git, GitHub (kpubdata-lab 조직: kpubdata, kpubdata-builder, kpubdata-studio)
	의사소통관리	GitHub Issues·Pull Request 메신저·회의 도구 기재
	기타사항	GitHub Actions: CI, 보안 점검, 이미지 빌드, 정기 수집, 계약 호환 점검

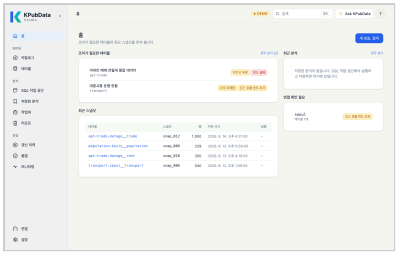
5. 장비(기자재/재료) 활용

번호	품명	작품에서의 주요기능
1	개발용 컴퓨터	Builder·Studio 개발과 시연 기종 기재
2	서버 또는 클라우드	배포 환경이 있으면 기재

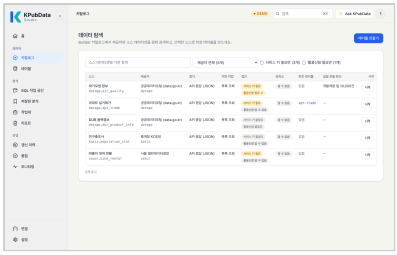
6. 프로젝트 작동 동영상

- 시연 영상 유튜브 링크 기재

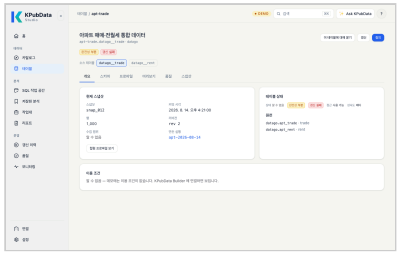
7. 결과물 상세 이미지



홈: 조치가 필요한 테이블과 최근 스냅샷



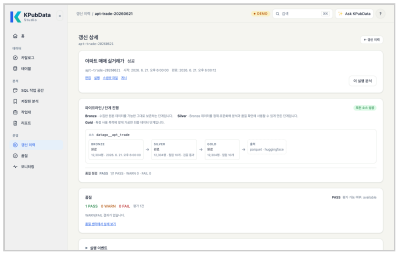
카탈로그: 소스 데이터셋 비교



테이블 상세: 스냅샷과 스키마



SQL 작업 공간: 스냅샷 질의



실행 상세: 단계별 진행과 품질



품질: 검사 결과와 스키마 변화

8. 달성 성과

☐ 논문게재 및 포스터발표	게재(발표)자명	논문(포스터)명	게재(발표)처	게재(발표)일자
☐ 앱(APP) 등록	등록자명	앱(APP)명	등록처	등록일자
☐ 프로그램 등록	등록자명	프로그램명	등록처	등록일자
☐ 특허/실용신안 출원	출원자명	특허/실용신안명	출원번호	출원일자
☐ 기술이전	기술이전기업명	기술명	금액	이전일자
☐ 공모전	구분(교내/대외)	공모전명	수상여부(출품/수상)	상격
☐ 실용화				
☐ 기타				

- 해당 성과를 체크하고 내용 기재. 증빙은 별첨1에 첨부

개발 결과 요약

구분	내용
공개 저장소	kpubdata, kpubdata-builder, kpubdata-studio (GitHub kpubdata-lab 조직)
Builder	HTTP API 77개 연산 (계약 1.123.0), CLI 명령 20여 종, 내보내기 6종, 게시 대상 3종
Studio	화면 20종, 한국어·영어, 데모 모드 (버전 0.4.0)
테스트	Builder 테스트 파일 285개, Studio 단위 테스트 파일 308개와 E2E 스펙 26개
정기 수집	기상, 부동산 실거래, 관광, 의약품 안전 사용 4개 분야를 일정에 따라 수집·게시

Ⅲ. 프로젝트 수행 내용

1. 업무분장

번호	성명	역할	담당업무
1		멘토	기재
2		지도교수	없으면 공란
3		팀장	기재
4		팀원2	기재
5		팀원3	기재
6		팀원4	기재
7		팀원5	기재

2. 프로젝트 수행일정 수행 기간을 노란색 셀로 표시

구분	추진내용	수행일정								
		3월	4월	5월	6월	7월	8월	9월	10월	11월
계획	주제 선정, 문제 정의, 수행계획서 작성									
분석	공공 API와 유사 제품 조사, 요구사항 정의									
설계	BuildSpec·Manifest 계약, 단계별 처리 구조, API 계약									
	메뉴·화면 설계, 제작설계서 작성									
개발	Builder 파이프라인, 품질·개인정보 검사									
	웨어하우스 스냅샷, SQL 질의, 내보내기·게시									
	Studio 화면 (카탈로그, 테이블, SQL, 리포트, 운영)									
	Ask KPubData, 인증, 다국어									
테스트	단위·통합·화면 테스트, 실연동 점검									
종료	결과보고서 작성, 시연 영상 제작									
온·오프라인 미팅	멘토링·팀 회의 기재									

3. 프로젝트 추진 과정에서의 문제점 및 해결방안

1) 프로젝트 관리 측면

- 일정·협업에서 겪은 문제와 해결 과정 기재

2) 프로젝트 개발 측면

- **대용량 데이터 처리 한계**
  - 문제: 약 1.4GB 파일 소스의 빌드가 실패 (945MB는 성공)
  - 원인: 업로드 본문을 통째로 다루고, 수집·정제·패키징·내보내기·질의의 다섯 지점에서 각각 전체 데이터를 메모리에 올리는 구조
  - 해결: 표 처리 엔진을 Polars에서 DuckDB로 바꾸고 단계 사이를 Parquet 파일로 연결. 전환 중에는 두 엔진의 결과를 비교하는 테스트로 동등성 확인
  - 결과: 실행 엔진 코드에서 Polars 의존을 제거. 기존 게시 스크립트만 이전 엔진을 유지하며 순차 이전 중
- **실패한 실행의 상태 표시**
  - 문제: 대기 중 취소되거나 서버 재시작으로 끝난 실행은 기록이 남지 않고, 테이블 커밋에 실패한 실행이 재시작 뒤 성공으로 보임
  - 원인: Manifest가 만들어지기 전에 끝난 실행은 남는 기록이 없고, 상태를 읽는 코드가 여러 곳에 흩어져 있었음
  - 해결: 종료 기록을 따로 남기고, 실행 결과(ok)와 API 작업 상태(succeeded)를 한 함수에서 판정. 테이블 커밋이 실패한 실행은 failed로 표시
  - 결과: 모든 실행이 하나의 상태를 가지며 인덱스를 다시 만들어도 같은 결과가 나옴
- **테이블 동시 갱신**
  - 문제: 같은 테이블을 두 실행이 동시에 갱신하면 나중 결과가 먼저 결과를 덮어쓸 수 있음
  - 원인: 커밋 시점에 다른 실행의 커밋 여부를 확인하지 않음
  - 해결: 실행 시작 때 테이블의 revision을 기억했다가 커밋할 때 비교하고, 다르면 거부. 커밋은 모든 소스가 끝난 뒤 한 스레드에서 수행
  - 결과: 충돌한 실행은 conflict 사유가 Manifest에 남고 API에서 failed로 표시되며 기존 스냅샷이 유지됨
- **AI 답변의 신뢰성**
  - 문제: LLM은 존재하지 않는 테이블이나 실행을 근거로 답하거나 실행할 수 없는 SQL을 제안할 수 있음
  - 원인: LLM 출력은 형식과 내용 모두 보장되지 않음
  - 해결: 응답을 구조 검증, 근거 대조, Builder 검증, 사용자 승인의 4단계로 처리. 근거에 없는 항목은 그 항목만 제거
  - 결과: SQL과 명세 변경은 사용자가 승인한 뒤에만 실행되며, SQL은 Builder의 읽기 전용 검사를 다시 거침

- **화면 구조와 용어**
  - 문제: 초기 메뉴가 Discover, Add Data, Builds 등 실행 엔진의 용어 중심으로 나뉘어 있었음
  - 원인: 화면 이름이 빌드, 데이터셋, provider 같은 Builder의 내부 개념을 그대로 따름
  - 해결: 카탈로그 → 테이블 → 분석 → 운영 순으로 메뉴를 다시 묶고, 테이블 만들기를 메뉴가 아닌 동작으로 바꿔 3단계로 줄임. 이전 주소는 새 주소로 자동 이동
  - 결과: 이전 화면 이름이 문구에 다시 들어오면 CI가 실패하도록 검사를 추가
- **화면에 보이는 값의 정확성**
  - 문제: Builder가 주지 않는 값을 0이나 임의 점수로 보여 주면 실제 상태로 오해할 수 있음
  - 원인: 이전 홈은 지표 카드 4종 중심이었는데 그중 일부는 Builder가 제공하지 않는 값이었음
  - 해결: 홈에서 지표 카드를 없애고 조치가 필요한 테이블을 먼저 표시. 품질은 점수 대신 검사 결과만 표시하고 받지 못한 값은 '확인 불가'로 표기
  - 결과: 모든 값이 Builder 응답에서 오며 응답 형식이 계약과 다르면 오류로 처리

3) 테스트 수행

대상	종류	내용
Builder	단위·통합 테스트	pytest. 테스트 파일 285개 (단위 271, 통합 7, 파이프라인 6, 엔진 동등성 1)
	정적 검사	ruff, mypy
	실행 환경	Python 3.10~3.13 각각에서 실행
Studio	단위 테스트	Vitest. 테스트 파일 308개. 커버리지 기준: 구문 84%, 분기 75%, 함수 85%, 줄 86%
	E2E 테스트	Playwright. 스펙 26개 (데스크톱, 모바일, 운영 빌드)
	화면 기준 이미지	핵심 5개 화면 × 3종 = 15장을 매 실행마다 비교
	접근성	axe-core로 화면별 검사
연동	계약 호환	Builder 계약과 Studio 스키마의 불일치를 CI에서 검사

- 이 보고서 작성 시점에 Studio의 리포트·AI 질의·홈 테스트 37개 통과와 운영 빌드 성공을 직접 확인
- Builder 테스트는 의존성 내려받기 오류로 이번에 직접 실행하지 못함 CI 결과 확인 후 수정

4. 프로젝트를 통해 배우거나 느낀 점

- 팀원별 소감 기재

Ⅳ. 기대효과 및 활용분야

1. 프로젝트의 기대효과

1) 데이터 이용자

- 기관마다 다른 형식을 표준 테이블로 맞추는 반복 작업 감소
- 가공 과정이 명세와 실행 기록으로 남아 누구나 같은 결과를 다시 만들 수 있음
- 코드를 작성하지 않아도 화면에서 테이블 생성과 SQL 분석 가능

2) 데이터 품질과 신뢰

- 품질·개인정보·라이선스 확인이 실행 과정에 포함되어 배포 전 점검 누락 방지
- 게시되는 데이터셋에 출처, 처리 내역, 스키마가 카드로 함께 제공됨
- 스냅샷으로 분석 시점의 입력이 고정되어 결과를 검증할 수 있음

3) 운영과 확장

- 별도 DB 서버 없이 SQLite와 파일만으로 동작해 설치와 운영 부담이 작음
- 내보내기 형식과 게시 대상을 플러그인으로 추가 가능
- 공개 저장소로 제공되어 누구나 내려받아 사용하고 확장할 수 있음

2. 프로젝트의 활용분야

분야	활용 방법	예시
교육	공공데이터 기반 수업 자료와 실습용 테이블 구축	지역별 인구·교통 데이터로 SQL 실습
연구	출처와 처리 내역이 기록된 연구용 데이터셋 구축	부동산 실거래 매매·전월세 통합 테이블
데이터 분석	주기적으로 갱신되는 테이블과 리포트 작성	기상 예보, 관광 정보의 정기 갱신
AI 개발	학습용 데이터셋 구축과 허브 배포	분할(train·valid·test)과 카드가 포함된 데이터셋을 Hugging Face에 게시
공공·민간 서비스	공공데이터를 쓰는 서비스의 데이터 준비 단계	의약품 안전 사용(DUR) 정보의 주 단위 갱신

Ⅴ. 참고자료

1. 참고 및 인용자료

- KPubData Builder 저장소: github.com/kpubdata-lab/kpubdata-builder
- KPubData Studio 저장소: github.com/kpubdata-lab/kpubdata-studio
- kpubdata 저장소: github.com/kpubdata-lab/kpubdata
- 공공데이터포털: www.data.go.kr
- DuckDB 문서: duckdb.org/docs
- Grand View Research, AI Training Dataset Market (2025)
- SW개발/HW제작 설계서 (26_HC277), 2026. 10. 12 개정

별첨1 2026년 한이음 드림업 프로젝트 성과 증빙자료

- 프로그램 등록 증명서, 논문·학술대회 초록집 등 성과 증빙 이미지 첨부